

Clase OrdenServicio

Tabla de propiedades

Propiedad	Tipo	Visibilidad	Propósito
numeroOrden	int	private	Almacenar el número que identifica de manera única la orden.
nombrePropietario	String	private	Almacenar el nombre del propietario del vehículo.
placaVehiculo	String	private	Almacenar la placa del vehículo relacionado con la orden.
descripcionServicio	String	private	Almacenar la descripción del mantenimiento o reparación solicitada.
costoEstimado	double	private	Almacenar el costo estimado del servicio.

Tabla de métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
OrdenServicio()	-	int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado	public	Construir e inicializar una orden de servicio con la información proporcionada.
getNumeroOrden()	int	Ninguno	public	Obtener el número de la orden.
getNombrePropietario()	String	Ninguno	public	Obtener el nombre del propietario.
getPlacaVehiculo()	String	Ninguno	public	Obtener la placa del vehículo.
getDescripcionServicio()	String	Ninguno	public	Obtener la descripción del servicio.



getCostoEstimado()	double	Ninguno	public	Obtener el costo estimado de la orden.
setDescripcionServicio()	void	String descripcionServicio	public	Modificar la descripción del servicio, validando que no esté vacía.
setCostoEstimado()	void	double costoEstimado	public	Modificar el costo estimado, validando que sea mayor que cero.

Clase ControladorOrdenes

Tabla de propiedades

Propiedad	Tipo	Visibilidad	Propósito
ordenes	List<OrdenServicio>	private	Almacenar dinámicamente todas las órdenes de servicio activas.

Tabla de Métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
ControladorOrdenes()	-	Ninguno	public	Construir el controlador e inicializar la colección con un ArrayList<OrdenServicio> vacío.
registrarOrden()	void	OrdenServicio orden	public	Verificar que el número no esté repetido y agregar la orden a la colección.
obtenerOrdenes()	List<OrdenServicio>	Ninguno	public	Obtener la colección con todas las órdenes registradas.
buscarOrden()	OrdenServicio	int numeroOrden	public	Buscar y devolver una orden utilizando su número.
modificarOrden()	void	int numeroOrden, String nuevaDescripcion, double nuevoCosto	public	Buscar una orden y modificar su descripción y costo estimado.
cancelarOrden()	void	int numeroOrden	public	Buscar una orden y eliminarla de la colección dinámica.
buscarOrdenesPorPlaca()	List<OrdenServicio>	String placaBuscada	public	Buscar y devolver todas las órdenes asociadas con una placa.
calcularCostoTotal()	double	Ninguno	public	Recorrer la colección y sumar los costos de las órdenes activas.



calcularCostoPromedio()	double	Ninguno	public	Calcular el costo promedio de las órdenes activas.
obtenerOrdenMayorCosto()	OrdenServicio	Ninguno	public	Buscar y devolver la orden que tenga el costo estimado más alto.
obtenerCantidadOrdenes()	int	Ninguno	public	Obtener la cantidad de órdenes almacenadas actualmente.
existeNumeroOrden()	boolean	int numeroOrden	private	Comprobar si un número de orden ya se encuentra registrado.

Clase VistaTaller

Tabla de propiedades

Propiedad	Tipo	Visibilidad	Propósito
lector	Scanner	private	Leer los datos ingresados por el usuario desde la consola.

Tabla de Métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
VistaTaller()	-	Ninguno	public	Construir la vista e inicializar el lector con System.in.
mostrarMenu()	int	Ninguno	public	Mostrar las diez opciones del menú y devolver la opción seleccionada.
leerTexto()	String	String mensaje	public	Mostrar un mensaje y leer un dato de texto.
leerEntero()	int	String mensaje	public	Mostrar un mensaje, leer un texto y convertirlo a número entero.
leerDecimal()	double	String mensaje	public	Mostrar un mensaje, leer un texto y convertirlo a número decimal.
mostrarMensaje()	void	String mensaje	public	Mostrar un mensaje informativo, de confirmación o de error.

mostrarOrden()	void	OrdenServicio orden	public	Mostrar toda la información de una orden.
mostrarOrdenes()	void	List<OrdenServicio> ordenes	public	Recorrer y mostrar las órdenes recibidas.
mostrarReporteCostos()	void	double total, double promedio	public	Mostrar el costo total y el costo promedio de las órdenes.
cerrarLector()	void	Ninguno	public	Cerrar el lector cuando termine la ejecución del programa.

Clase Main

Tabla de Propiedades

-

Tabla de Métodos

Método	Retorno	Parámetros	Visibilidad	Propósito
main()	void	String[] args	public static	Crear la vista y el controlador, mostrar el menú y coordinar todas las operaciones del sistema.

Análisis Preguntas

¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección guardará las órdenes de servicio. Cada orden tendrá su número, nombre del propietario, placa, descripción del servicio y costo estimado.

¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los datos de cada orden serán solicitados al usuario y enviados al constructor. La lista de órdenes empezará vacía y los valores utilizados para hacer las sumas comenzarán en cero.

¿Cómo identificará una orden específica dentro de la colección?

Cada orden se identificará por su número de orden, el cual no se podrá repetir. Se recorrerá la lista hasta encontrar una orden que tenga el mismo número ingresado.

¿Cómo agregará y eliminará órdenes de la colección dinámica?

Antes de agregar una orden se comprobará que su número no esté repetido. Para eliminarla, primero se buscará por su número y después se quitará de la lista.

¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se recorrerá toda la lista comparando la placa ingresada con la placa de cada orden. Las órdenes que tengan la misma placa se guardarán en otra lista para después mostrarlas.

¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Se utilizará un ciclo para recorrer las órdenes y sumar sus costos. El promedio se calculará dividiendo el total entre la cantidad de órdenes. Si no hay órdenes, el promedio será cero.

¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se tomará la primera orden como la de mayor costo y después se comparará con las demás. Si se encuentra una orden con un costo mayor, esta pasará a ser la nueva orden de mayor costo.

¿Qué situaciones dentro del programa pueden producir una excepción?

Puede ocurrir una excepción si se ingresan letras cuando se espera un número, si se dejan datos vacíos, si el costo no es válido, si el número de orden está repetido o si se intenta buscar, modificar o cancelar una orden que no existe.

¿Qué operaciones serán protegidas utilizando try-catch?

Se utilizará try-catch al leer números, registrar órdenes y realizar búsquedas, modificaciones o cancelaciones. Esto evitará que el programa termine cuando el usuario ingrese un dato incorrecto.

¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

El bloque finally se utilizará para mostrar el mensaje "Operación de búsqueda finalizada". Este mensaje aparecerá tanto si la búsqueda fue correcta como si ocurrió un error.